



Automation

CI/CD Fundamentals



Outline

- 1 Pendahuluan**
- 2 Pipeline dalam CI/CD**
- 3 Hands-on**

Memahami Konsep CI/CD

Mempelajari prinsip dasar Continuous Integration dan Continuous Deployment serta manfaatnya dalam pengembangan perangkat lunak



Mengenal Jenkins

Memahami cara kerja Jenkins sebagai alat otomatisasi untuk pipeline CI/CD



Objective

Implementasi Praktis

Belajar cara membuat dan mengkonfigurasi pipeline CI/CD menggunakan Jenkins melalui hands-on



Optimalisasi Alur Kerja

Menerapkan praktik terbaik untuk mengoptimalkan alur kerja pengembangan perangkat lunak



1

Pendahuluan

CI/CD Fundamentals



Apa itu Continuous Integration (CI)?



Definisi CI

Proses otomatis untuk mengintegrasikan perubahan kode secara terus-menerus ke dalam repository utama, memastikan kode selalu dalam keadaan siap diuji.



Fokus CI

Menjaga kualitas kode, mempercepat deteksi bug, dan memastikan integrasi kode berjalan mulus antar tim pengembang.



Manfaat CI

- Deteksi masalah lebih awal
- Mengurangi konflik dalam integrasi kode
- Mengurangi waktu untuk merilis fitur baru

Langkah-langkah dalam CI

Commit

Pengembang mengirimkan perubahan kode ke repositori bersama

Build

Server CI (Jenkins) membangun aplikasi secara otomatis

Test

Pengujian otomatis dilakukan untuk memastikan tidak ada bug baru

Feedback

Tim mendapatkan umpan balik tentang kualitas kode

Apa itu Continuous Deployment (CD)?



Definisi CD

Proses otomatis untuk mengirimkan perubahan kode ke lingkungan produksi secara terus-menerus setelah melewati pengujian CI, tanpa intervensi manual.



Fokus CD

Memastikan aplikasi selalu dalam keadaan siap produksi dan dapat dirilis kapan saja dengan risiko minimal.



Manfaat CD

- Penyebaran lebih cepat dan lebih sering
- Mengurangi risiko kesalahan manual
- Mempercepat waktu untuk mendapatkan umpan balik dari pengguna

Langkah-langkah dalam CD

Build

Kode dibangun dalam lingkungan pengujian yang terkontrol

Test

Kode yang sudah dibangun diuji secara menyeluruh dengan berbagai jenis pengujian

Staging

Aplikasi di deploy ke lingkungan staging yang mirip dengan produksi

Deploy

Jika semua pengujian berhasil, aplikasi di deploy ke produksi secara otomatis

Perbedaan CI dan CD

Continuous Integration (CI)

- Fokus pada integrasi dan pengujian kode
- Meningkatkan kualitas kode secara keseluruhan
- Diterapkan di tahap pengembangan
- Memastikan kode baru terintegrasi dengan baik
- Mendeteksi masalah lebih awal dalam siklus pengembangan

Continuous Deployment (CD)

- Fokus pada pengiriman dan penerapan kode ke produksi
- Menjamin aplikasi selalu dalam keadaan siap produksi
- Diterapkan di tahap produksi
- Mengotomatisasi proses rilis
- Mengurangi waktu antara pengembangan dan penggunaan



2

Pipeline dalam CI/CD

CI/CD Fundamentals

Apa itu Pipeline dalam CI/CD?

Pipeline dalam CI/CD adalah alur otomatis yang terdiri dari serangkaian langkah yang digunakan untuk melakukan build, test, dan deploy aplikasi. Pipeline memastikan setiap perubahan kode melewati serangkaian pengujian dan validasi sebelum sampai ke produksi.

Otomatisasi

Pipeline mengotomatiskan seluruh proses dari commit kode hingga deployment, mengurangi intervensi manual dan risiko kesalahan.

Konsistensi

Setiap perubahan kode melewati proses yang sama, memastikan konsistensi dalam pengujian dan deployment.

Visibilitas

Pipeline memberikan visibilitas ke seluruh proses, memudahkan tim untuk melacak status dan mengidentifikasi masalah.

Komponen Pipeline CI/CD

Source Stage

Mengambil kode sumber dari repositori

Build Stage

Proses kompilasi dan pembuatan aplikasi

Test Stage

Pengujian otomatis, seperti unit test dan integrasi

Deploy Stage

Pengiriman aplikasi ke server staging atau produksi

Jenis Pipeline dalam Jenkins

Declarative Pipeline

Declarative Pipeline adalah pendekatan yang lebih terstruktur dan mudah dibaca untuk mendefinisikan pipeline CI/CD di Jenkins. Pipeline ini menggunakan sintaks yang jelas dan didefinisikan dengan menggunakan blok pipeline {} dalam Jenkinsfile.

- Mudah dibaca dan ditulis
- Struktur yang jelas dan terorganisir
- Cocok untuk pipeline standar

Scripted Pipeline

Scripted Pipeline menawarkan fleksibilitas yang lebih tinggi dengan menggunakan skrip Groovy untuk mendefinisikan langkah-langkah pipeline. Pendekatan ini memberikan kontrol yang lebih besar terhadap alur proses.

- Lebih fleksibel dan kuat
- Menggunakan skrip Groovy
- Cocok untuk pipeline kompleks

Visualisasi Pipeline CI/CD

Commit to Git

Pengembang mengirim perubahan kode ke repositori Git

Build & Test

Jenkins membangun dan menguji aplikasi

Deploy ke Staging

Aplikasi di deploy ke lingkungan staging

Deploy ke Produksi

Aplikasi di deploy ke lingkungan produksi

Jenkins dan **CI/CD Pipeline**



Otomatisasi

Jenkins mengotomatiskan seluruh proses CI/CD, dari integrasi kode hingga deployment ke produksi.



Ekosistem Plugin

Jenkins memiliki ribuan plugin yang memungkinkan integrasi dengan berbagai alat dan teknologi.



Antarmuka Pengguna

Jenkins menyediakan dashboard yang intuitif untuk mengelola dan memantau pipeline CI/CD.



Integrasi

Jenkins terintegrasi dengan berbagai alat seperti Git, Docker, dan Kubernetes.

3

Hands-on

CI/CD Fundamentals



Persiapan Lingkungan untuk Hands-On



Instalasi Jenkins

Memastikan Jenkins sudah terpasang pada sistem, baik secara lokal atau di server. Jenkins dapat diakses melalui web browser, biasanya di alamat `http://localhost:8080`.



Menyiapkan Git Repository

Membuat repository Git sederhana (misalnya di GitHub atau GitLab) untuk proyek yang akan digunakan dalam pipeline CI/CD.



Menyiapkan Kode Aplikasi

Menyiapkan kode aplikasi sederhana yang akan dibangun dan diuji dalam pipeline Jenkins, sebagai bahan praktik.

Instalasi Jenkins

Unduh Jenkins

Mengunduh installer Jenkins dari situs resmi jenkins.io sesuai dengan sistem operasi yang digunakan (Windows, macOS, atau Linux).

Instalasi

Menjalankan installer dan mengikuti petunjuk instalasi. Pastikan Java sudah terpasang di sistem karena Jenkins membutuhkan Java untuk berjalan.

Konfigurasi Awal

Membuka Jenkins di browser dan menyelesaikan konfigurasi awal, termasuk memasukkan password admin dan menginstal plugin yang direkomendasikan.

Menyiapkan **Git Repository**

Buat Akun Git

Jika belum memiliki akun, buat akun di platform Git seperti GitHub, GitLab, atau Bitbucket.

Buat Repository Baru

Buat repository baru di platform Git pilihan Anda untuk menyimpan kode proyek.

Clone Repository

Clone repository ke komputer lokal Anda untuk mulai bekerja dengan kode.

Push Kode Awal

Tambahkan kode aplikasi sederhana ke repositori dan push ke remote repository.

Menyiapkan Kode **Aplikasi Sederhana**

Struktur Proyek

Buat struktur proyek sederhana yang mencakup kode sumber, file konfigurasi, dan skrip build. Untuk contoh Java, struktur proyek Maven standar adalah pilihan yang baik.

- `src/main/java`: Kode sumber utama
- `src/test/java`: Kode pengujian
- `pom.xml`: File konfigurasi Maven

Aplikasi Contoh

Buat aplikasi sederhana yang dapat dibangun dan diuji. Misalnya, aplikasi Java sederhana dengan beberapa metode dan unit test untuk mendemonstrasikan pipeline CI/CD.

Pastikan aplikasi dapat dibangun dengan perintah build standar (misalnya 'mvn clean install' untuk proyek Maven) dan memiliki pengujian otomatis.

Membuat Job Pipeline Baru di Jenkins



Akses Dashboard Jenkins

Buka browser dan akses dashboard Jenkins, biasanya di alamat `http://localhost:8080`. Masuk dengan kredensial admin yang telah dibuat saat instalasi.



Buat Item Baru

Klik "New Item" di dashboard Jenkins. Beri nama job pipeline (misalnya "MyAppPipeline") dan pilih tipe "Pipeline" dari daftar opsi.



Konfigurasi Pipeline

Pada halaman konfigurasi, atur sumber kode dari Git repository. Masukkan URL repository dan kredensial jika diperlukan untuk mengakses repository.

Menulis Jenkinsfile Dasar

Jenkinsfile adalah file yang mendefinisikan pipeline CI/CD dalam format yang dapat dibaca Jenkins. File ini biasanya disimpan di root repositori Git dan berisi definisi langkah-langkah yang akan dijalankan dalam pipeline.

```
pipeline { agent any stages { stage('Build') { steps { echo 'Building the application...' sh 'mvn clean install' } } stage('Test') { steps { echo 'Running tests...' sh 'mvn test' } } stage('Deploy') { steps { echo 'Deploying to production...' sh './deploy.sh' } } }}
```

Penjelasan Komponen **Jenkinsfile**

Pipeline

Blok utama yang mendefinisikan seluruh pipeline CI/CD. Semua konfigurasi pipeline berada di dalam blok ini.

Agent

Menentukan di mana pipeline akan dijalankan. 'any' berarti pipeline dapat dijalankan di agen Jenkins mana pun yang tersedia.

Stages

Berisi kumpulan stage yang mendefinisikan langkah-langkah dalam pipeline, seperti build, test, dan deploy.

Steps

Berisi perintah yang akan dijalankan dalam setiap stage, seperti perintah shell atau perintah Jenkins khusus.

Stage Build dalam Pipeline

Tujuan Stage Build

Stage Build bertujuan untuk mengompilasi kode sumber menjadi artefak yang dapat dijalankan. Pada aplikasi Java, ini berarti mengompilasi kode Java menjadi file JAR atau WAR.

Stage ini juga biasanya mencakup langkah-langkah seperti mengunduh dependensi, menjalankan linter untuk memeriksa kualitas kode, dan mempersiapkan artefak untuk pengujian.

Contoh Konfigurasi

```
stage('Build') { steps { echo 'Building the application...' sh 'mvn clean compile' sh 'mvn package -DskipTests' }}
```

Perintah 'mvn clean compile' membersihkan build sebelumnya dan mengompilasi kode sumber. Perintah 'mvn package -DskipTests' membuat paket aplikasi tanpa menjalankan pengujian (yang akan dilakukan di stage berikutnya).



Stage Test dalam Pipeline

Tujuan Stage Test

Stage Test bertujuan untuk menjalankan berbagai jenis pengujian untuk memastikan kualitas kode. Ini mencakup unit test, integration test, dan kadang-kadang juga performance test atau security test.

Pengujian yang berhasil adalah prasyarat untuk melanjutkan ke stage berikutnya, memastikan hanya kode berkualitas tinggi yang dideploy ke produksi.

Contoh Konfigurasi

```
stage('Test') { steps { echo 'Running tests...' sh 'mvn test' junit '**/target/surefire-reports/*.xml' }}
```

Perintah 'mvn test' menjalankan semua unit test dalam proyek. Perintah 'junit' mengumpulkan hasil pengujian dalam format XML dan menampilkannya di dashboard Jenkins.

Stage Deploy dalam Pipeline

Tujuan Stage Deploy

Stage Deploy bertujuan untuk mengirimkan aplikasi yang telah dibangun dan diuji ke lingkungan target, seperti server staging atau produksi. Ini adalah langkah terakhir dalam pipeline CI/CD.

Deployment dapat dilakukan dengan berbagai cara, tergantung pada jenis aplikasi dan infrastruktur target, seperti deployment ke server fisik, container Docker, atau platform cloud.

Contoh Konfigurasi

```
stage('Deploy') { steps { echo 'Deploying to production...' sh  
'./deploy.sh' sh 'curl -X POST http://target-server/deploy' }}
```

Skrip './deploy.sh' adalah skrip kustom yang berisi logika deployment. Perintah 'curl' mengirimkan permintaan HTTP ke server target untuk memicu deployment.



Menjalankan Pipeline di Jenkins

Akses Job Pipeline

Dari dashboard Jenkins, klik pada job pipeline yang telah dibuat sebelumnya untuk membuka halaman detail job.

Mulai Build

Klik tombol "Build Now" di menu sebelah kiri untuk memulai eksekusi pipeline. Jenkins akan mengambil kode dari repositori Git dan menjalankan langkah-langkah yang didefinisikan dalam Jenkinsfile.

Pantau Progres

Pantau progres pipeline melalui tampilan Stage View yang menunjukkan status setiap stage (Build, Test, Deploy) dengan indikator warna (hijau untuk sukses, merah untuk gagal).

Memverifikasi Hasil Pipeline

Periksa Status Build

Pastikan status build keseluruhan adalah sukses (ditandai dengan warna hijau). Jika ada stage yang gagal, status build akan ditandai sebagai gagal (warna merah).

Tinjau Log Build

Klik pada nomor build untuk melihat log detail. Periksa log untuk memahami apa yang terjadi selama eksekusi pipeline dan mengidentifikasi masalah jika ada.

Periksa Hasil Test

Tinjau hasil pengujian untuk memastikan semua test telah dijalankan dan berhasil. Jenkins biasanya menyediakan laporan pengujian yang detail.

Verifikasi Deployment

Pastikan aplikasi telah berhasil di deploy ke lingkungan target dengan mengakses aplikasi dan memverifikasi fungsionalitasnya.

Menangani Masalah dalam Pipeline



Masalah Build

Jika stage Build gagal, periksa log untuk kesalahan kompilasi, masalah dependensi, atau masalah konfigurasi build. Pastikan semua dependensi tersedia dan kode dapat dikompilasi secara lokal.



Kegagalan Test

Jika stage Test gagal, identifikasi test yang gagal dari laporan pengujian. Perbaiki kode atau test yang bermasalah, dan jalankan test secara lokal untuk memverifikasi perbaikan sebelum push ke repository.



Masalah Deployment

Jika stage Deploy gagal, periksa konektivitas ke server target, kredensial, dan konfigurasi deployment. Pastikan skrip deployment berfungsi dengan benar dan server target siap menerima deployment.

Menggunakan Parameter dalam Pipeline

Apa itu Parameter Pipeline?

Parameter pipeline memungkinkan Anda untuk menyesuaikan perilaku pipeline saat runtime. Parameter dapat digunakan untuk menentukan lingkungan deployment, versi aplikasi, atau konfigurasi lainnya tanpa mengubah kode pipeline.

Jenkins mendukung berbagai jenis parameter, seperti string, boolean, choice, dan file parameter, yang dapat dikonfigurasi dalam Jenkinsfile.

Contoh Konfigurasi

```
pipeline { agent any parameters {  
    string(name: 'ENVIRONMENT', defaultValue:  
    'staging', description: 'Deployment  
environment')    choice(name: 'VERSION',  
    choices: ['1.0.0', '1.1.0', '1.2.0'], description:  
    'Version to deploy')    booleanParam(name:  
    'RUN_TESTS', defaultValue: true, description:  
    'Run tests before deployment') } stages {  
    // Stages menggunakan parameter }}
```

Menggunakan Parameter dalam Stage

Parameter yang didefinisikan dalam pipeline dapat digunakan dalam stage untuk mengontrol alur eksekusi atau menyesuaikan perilaku pipeline berdasarkan nilai parameter.

```
stage('Test') { when { expression { params.RUN_TESTS == true } } steps {  
  echo "Running tests before deployment to ${params.ENVIRONMENT}" sh 'mvn test'  
}}stage('Deploy') { steps { echo "Deploying version ${params.VERSION} to  
${params.ENVIRONMENT}" sh "./deploy.sh ${params.ENVIRONMENT}  
${params.VERSION}" }}
```

Dalam contoh di atas, stage Test hanya akan dijalankan jika parameter RUN_TESTS bernilai true. Stage Deploy menggunakan parameter ENVIRONMENT dan VERSION untuk menyesuaikan perintah deployment.

Parallel Execution dalam Pipeline

Apa itu Parallel Execution?

Parallel execution memungkinkan beberapa task dalam pipeline dijalankan secara bersamaan, menghemat waktu eksekusi pipeline secara keseluruhan. Ini sangat berguna untuk menjalankan pengujian yang memakan waktu atau task yang tidak bergantung satu sama lain.

Jenkins mendukung parallel execution dalam stage dengan menggunakan blok 'parallel' yang berisi beberapa task yang akan dijalankan secara bersamaan.

Contoh Konfigurasi

```
stage('Test') { parallel {
  stage('Unit Tests') {
    steps {
      echo 'Running unit tests...'
      sh 'mvn test'
    }
  }
  stage('Integration Tests') {
    steps {
      echo 'Running integration tests...'
      sh 'mvn verify'
    }
  }
  stage('Performance Tests') {
    steps {
      echo 'Running performance tests...'
      sh './performance-test.sh'
    }
  }
}
```

Post Actions dalam Pipeline

Post actions memungkinkan Anda untuk menjalankan langkah-langkah tambahan setelah stage atau seluruh pipeline selesai, berdasarkan hasil eksekusi (sukses, gagal, atau selalu).

```
pipeline { agent any stages { stage('Build') { steps {  
    echo 'Building the application...' sh 'mvn clean install'  
} post { success { echo 'Build successful!  
Archiving artifacts...' archiveArtifacts artifacts:  
'target/*.jar', fingerprint: true } failure {  
echo 'Build failed! Sending notification...' mail to:  
'team@example.com', subject: 'Build Failed', body: 'The build has  
failed.' } } } } post { always { echo  
'Cleaning up workspace...' cleanWs() } }}
```



Menggunakan **Environment Variables**

Apa itu Environment Variables?

Environment variables adalah variabel yang tersedia dalam pipeline untuk menyimpan dan mengakses informasi seperti versi build, branch Git, atau konfigurasi lainnya. Jenkins menyediakan banyak environment variables bawaan, dan Anda juga dapat mendefinisikan variabel kustom.

Environment variables dapat diakses dalam pipeline menggunakan sintaks `${env.VARIABLE_NAME}` dan dapat digunakan dalam perintah shell atau langkah-langkah pipeline lainnya.

Contoh Konfigurasi

```
pipeline { agent any environment {  
  APP_VERSION = '1.0.0'    DEPLOY_ENV =  
  'staging' } stages {    stage('Build') {  
    steps {                echo "Building version  
${env.APP_VERSION}..."    sh "mvn  
versions:set  
-DnewVersion=${env.APP_VERSION}"  
sh 'mvn clean install'      }    }  
stage('Deploy') {        steps {          echo  
"Deploying to ${env.DEPLOY_ENV}..."  
sh "./deploy.sh ${env.DEPLOY_ENV}"      }  
}  }}
```

Menggunakan Credentials dalam Pipeline



Apa itu Credentials?

Credentials adalah informasi sensitif seperti password, token API, atau kunci SSH yang digunakan dalam pipeline untuk mengakses sistem eksternal. Jenkins menyediakan sistem manajemen credentials yang aman untuk menyimpan dan menggunakan informasi ini dalam pipeline.



Jenis Credentials

- Username/Password: Untuk autentikasi dasar
- SSH Key: Untuk koneksi SSH ke server
- Secret Text: Untuk token API atau string rahasia lainnya
- Certificate: Untuk autentikasi berbasis sertifikat



Contoh Penggunaan

```
stage('Deploy') { steps {  
  withCredentials([  
    usernamePassword(  
      credentialsId:  
      'my-server-credentials',  
      usernameVariable: 'USERNAME',  
      passwordVariable: 'PASSWORD'  
    )  ]){  
    sh "sshpass -p  
$PASSWORD ssh  
$USERNAME@server ./deploy.sh"  
  }  
}
```

Menggunakan Multibranch Pipeline

Apa itu Multibranch Pipeline?

Multibranch pipeline adalah fitur Jenkins yang secara otomatis membuat dan menjalankan pipeline untuk setiap branch dalam repositori Git. Ini sangat berguna dalam pengembangan berbasis fitur, di mana setiap fitur dikembangkan dalam branch terpisah.

Jenkins akan secara otomatis mendeteksi branch baru, membuat pipeline untuk branch tersebut, dan menjalankan pipeline setiap kali ada perubahan di branch.

Keuntungan

- Otomatisasi untuk semua branch
- Isolasi pengembangan fitur
- Pengujian paralel untuk berbagai branch
- Integrasi yang lebih baik dengan alur kerja Git

Untuk menggunakan multibranch pipeline, buat job baru dengan tipe "Multibranch Pipeline" dan konfigurasikan sumber repositori Git. Jenkins akan secara otomatis mendeteksi Jenkinsfile di setiap branch dan membuat pipeline yang sesuai.

Menggunakan Blue Ocean

Dashboard Visual

Blue Ocean menyediakan dashboard visual yang menampilkan status pipeline dengan cara yang lebih intuitif, memudahkan pemantauan status build.

Pipeline Editor

Blue Ocean dilengkapi dengan editor pipeline visual yang memungkinkan Anda membuat dan mengedit pipeline tanpa menulis kode Jenkinsfile secara manual.

Branch dan Pull Request

Blue Ocean menyediakan tampilan yang lebih baik untuk multibranch pipeline dan pull request, memudahkan pengelolaan alur kerja berbasis Git.

Debugging Pipeline

Blue Ocean menyediakan alat debugging yang lebih baik untuk pipeline, memudahkan identifikasi dan perbaikan masalah dalam pipeline.

Integrasi Jenkins dengan Docker

Mengapa Menggunakan Docker dengan Jenkins?

Docker memungkinkan Anda untuk menjalankan build dan test dalam lingkungan yang terisolasi dan konsisten, menghindari masalah "works on my machine". Integrasi Jenkins dengan Docker memungkinkan pipeline CI/CD yang lebih handal dan portabel.

Docker juga memudahkan deployment aplikasi, karena aplikasi dapat dikemas dalam container yang dapat dijalankan di berbagai lingkungan tanpa perubahan konfigurasi.

Contoh Konfigurasi

```
pipeline { agent { docker { image  
'maven:3.8.4-openjdk-11' } } stages {  
stage('Build') { steps { sh 'mvn  
clean install' } } stage('Test') {  
steps { sh 'mvn test' } } } }
```

Dalam contoh di atas, pipeline akan dijalankan dalam container Docker dengan image Maven, memastikan lingkungan build yang konsisten.

Integrasi Jenkins dengan Kubernetes

Apa itu Kubernetes?

Kubernetes adalah platform orkestrasi container yang mengelola deployment, scaling, dan manajemen aplikasi dalam container. Integrasi Jenkins dengan Kubernetes memungkinkan pipeline CI/CD yang lebih skalabel dan efisien.

Jenkins Kubernetes Plugin

Plugin Kubernetes untuk Jenkins memungkinkan Jenkins untuk membuat pod Kubernetes dinamis sebagai agen Jenkins, meningkatkan skalabilitas dan efisiensi resource.

Deployment ke Kubernetes

Jenkins dapat digunakan untuk men-deploy aplikasi ke cluster Kubernetes, mengotomatiskan proses deployment dan memastikan konsistensi antara lingkungan.

Contoh Konfigurasi

```
pipeline { agent { kubernetes {  
  yaml """apiVersion: v1kind: Podspec:  
containers: - name: maven image:  
maven:3.8.4-openjdk-11 command: - cat  
tty: true""" } } stages { stage('Build')  
{ steps { container('maven') {  
sh 'mvn clean install' } } } }
```

Tantangan dalam **Implementasi CI/CD**

Perubahan Budaya

Mengadopsi CI/CD memerlukan perubahan budaya dalam organisasi, dari pengembangan tradisional ke pendekatan yang lebih otomatis dan kolaboratif.

Kompleksitas

Pipeline CI/CD dapat menjadi kompleks, terutama untuk aplikasi besar dengan banyak dependensi dan lingkungan deployment.

Pengujian

Menulis pengujian otomatis yang komprehensif dan handal dapat menjadi tantangan, tetapi sangat penting untuk CI/CD yang efektif.

Infrastruktur

Menyiapkan dan mengelola infrastruktur CI/CD yang skalabel dan handal memerlukan investasi dan keahlian.

A background image showing three business professionals in an office. A man in a suit and glasses is standing and smiling, with his arms raised in a celebratory gesture. Two women, also in business attire, are seated at a desk with a laptop, looking up at him with smiles. The entire scene is overlaid with a semi-transparent teal filter.

</Rakamin

Terima kasih